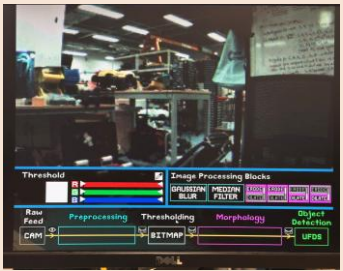
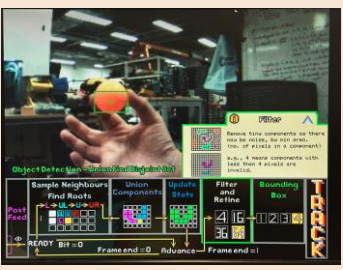
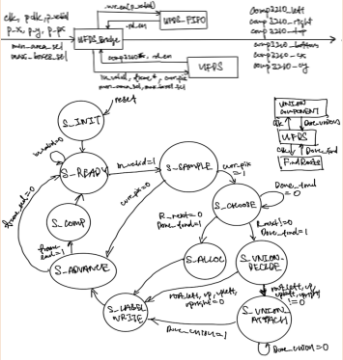
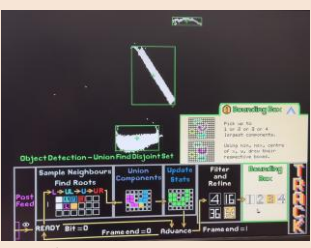
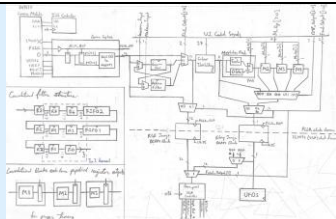
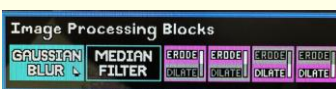
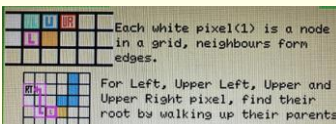
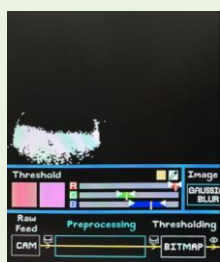


Student and Component Name	Component Descriptions	Images/Photos
Team S1-02 Create your own Object Tracker	An educational object tracking system that allows users to interactively build an image processing pipeline (preprocessing → threshold → morphology) and change views to see the effects, learn about the object detection algorithm and tune its parameters to see the bounding boxes, before running a servo with PD control for autonomous tracking of objects within the user's desired colour range. By testing the various options and witnessing real time results of their own actions/inputs, users can interactively learn how basic object tracking can be done. This system integrates the following hardware: 1x Basys3 FPGA, 1x OV7670 Camera, 2x MG90 Servo, 1x USB Mouse and 1x 640x480px VGA display.	
Kenneth Wong Cun Wi GUIs (Image Processing Settings and Object Detection Settings) Union-Find Disjoint Set (UFDS) Object Detection Algorithm	1. Menu: entry screen with title and prompt to start by inserting a physical coin. 2. Image Processing Settings GUI (bottom section over dimmed camera feed): - Apply image processing techniques (Gaussian Blur, Median Filter, Erode and Dilate) via drag-and-drop blocks into the pipeline, which snaps in place and distributes the blocks evenly about the centre. Blocks will snap back to selection bar if they are not placed in the rightful drop box (e.g. erode/dilate should be under morphology). Use scroll wheel to toggle between erode (up) and dilate (down) for each block. Image processing blocks - Click eye icons to switch between 4 different live views (Camera, Preprocessing output, Bitmap Morphology result) to witness the effects after each processing stage. - Users set threshold boundaries using sliders and eyedropper button. - Clicking the UFDS button will switch GUI to Object Detection Settings. 3. Object Detection Settings GUI (bottom section over dimmed camera feed): - Overlays with a state diagram of the UFDS object detection algorithm and draws the bounding boxes of detected objects, live view still runs in the background. - Each of the 5 states are clickable educational boxes that changes the content of the educational tab. Clicking the info tab animates vertically to open/close the info panel, which explains how each state work through texts and diagrams. - After learning the algorithm, users may set detection constraints (min area, number of boxes) using clickable boxes. - After which, clicking "TRACK" toggles the physical servo to start/stop autonomous object tracking. - Clicking "Post Feed" switches GUI back to Image Processing Settings. 4. Fullscreen Mode: Clears settings overlays and adds a crosshair to indicate centre of screen. Retains configurations from past settings. Right click in any state to enter full screen. Return to the settings page users were in previously by right clicking again. This module performs object detection using UFDS while streaming a binary image in raster order, extracting green object bounding boxes on the VGA display as the frame is processed. Since pixel source runs on pclk 25 MHz, while UFDS runs faster at 50Mhz, a UFDS Bridge and FIFO is used to decouple these domains and absorb the UFDS's variable per-pixel latency, as shown on the right. UFDS finds and connects a disjoint set of N objects efficiently, each is a tree of nodes with a root. In FPGA implementation, the algorithm is executed as shown in the overall UFDS diagram. The disjoint set data structure is achieved using register arrays to store essential data such as a <code>parent[label]</code>. A two-line buffer instead of the entire frame is used to significantly reduce needed memory space. Users may explore the Object Detection Settings GUI which is overlayed with a simplified UFDS state diagram and the five educational tabs will elaborate on the 5 main processes running on the backend. Filtering parameter <code>min_area</code> is exposed to users so they can experience how to filter out noises. Users should change view to post-bitmap mode on Image Processing Settings GUI first to see the noise (will be white dots that are not part of the object). Click one of the four boxes 4, 16, 32, 64 to change the minimum number of pixels to be considered a valid component. Higher number results in less noise. Choose n largest component to be bounded by clicking on $n = 1, 2, 3$ or 4.	 Image Processing Settings GUI  Object Detection Settings GUI  Fullscreen mode  Overall UFDS Diagram  Bounding boxes drawn over the 3 largest components

Tan Kai Cong Computer Vision Pipeline	Interfaced with the camera through its 8-bit parallel digital port with the configuration of its registers via its I2C bus. Implemented the general 3x3 kernel sliding operation that is used for all the convolutional image processing blocks (Gaussian Blur, Median Filter, Erode/Dilate) using a dual FIFO data stream to consume minimal memory. Pipeline registers are added to the output of each image processing block to ensure proper timing is met from cascading convolution operations. Setup overall architecture to connect user settings in the GUI to the underlying convolutional blocks to produce the correct order of CV operations. The RGB and bitmap image frames are saved into separate dual port buffers located in BRAM, allowing for safe clock domain crossing from the camera's generated 24 MHz clock (PCLK) to the VGA controller's required 25 MHz clock.	 Overall CV Architecture with convolution filter structure  Raw image (left) Gaussian blurred image (right)  Raw threshold bitmap (left) 2x dilated image (right)  Tuning mode for PD constants
Joel Ku VGA output display and text generation	The camera is mounted on a pan-tilt system. The object's X and Y pixel distance from the centre is used as the error to perform PID control with the pan and tilt servos respectively. To enable/disable the servo tracking, the user should navigate to the object detection settings page and left click on the "Track" button. The KP and KD constants for both pan and tilt servos can also be tuned. From left to right, each digit on the seven-segment display corresponds to the pan servo's 4-bit KP and KD value, then the tilt servo's KP and KD value. To change them, press BTNC. LED15 will light up, indicating PID is now in tuning mode. Each 4-bit group of switches correspond to the PD constant shown on the seven-segment from left to right (e.g. sw[15:12] and sw[11:8] is the pan servo's KP and KD value respectively). Pressing BTNC again exits tuning mode and sets the selected PD constants. Other servo functions tied to the buttons are pressing BTNR which resets the pan-tilt system to its home position, and pressing BTND to reset the PD constants to their pre-tuned default values.	 Pulled from BRAM  Image Processing Blocks Hover effect and scroll to switch graphics for erode/dilate  Text generated from generator
Joshua Yeo Wee Tze Image thresholding	As part of the computer vision/image processing pipeline described by Kaicong, the image needs to be threshold by a desired colour range to convert the RGB frame into a bitmap. Users will first use the eyedropper to obtain the corresponding RGB value they want. Click on the eyedropper button (it will be active with grey background), click on the object/colour to track. This will read and display current mouse position's RGB value in the camera feed. Then, small markers, filled with the eyedropped colour, will be positioned within each R, G and B channel to pinpoint the specific R, G and B values of the eyedropped colour. The colour will be reflected beside the eyedropper too. Users will now threshold the colours using the two arrows on the three RGB channels. Left click within the area of the arrows to drag and slide it. As colour channel spans a range of 160 pixel, sliders shift 10 pixel every step. Left sliders' x-positions determine lower bound of the threshold while right slider for upper bound. Hence, drag the 6 arrows to bound the colour accordingly. Left and right boxes beside the sliders will responsively display the left and right RGB extreme values based on slider position. When user clicks and holds the left mouse button on a slider, it stays active and follows the mouse's horizontal movement (x-axis) even if the cursor moves outside the slider area. During this time, other sliders will not respond until the click is released. Toggle the eye icon after BITMAP to see objects within the threshold on the screen. Mouse Cursor design to select boxes and screen boundary.	 Eyedropper in CAM view and move sliders to bound colours  Thresholded image on BITMAP view, shows the object only.

S1-02:	Joel Ku A0308792U	Joshua Yeo Wee Tze A0309329Y	Kenneth Wong Cun Wi A0303203A	Tan Kai Cong A0309057A
---------------	--------------------------	-------------------------------------	--------------------------------------	-------------------------------

References:

The following papers have inspired and supported the computer vision pipeline:

1. Wang, B., & Xiang, Q. (2020). Fast median filter image processing algorithm and its FPGA implementation. *Frontiers in Signal Processing*, 4(4). <https://doi.org/10.22606/fsp.2020.44002>
2. Campos, N. (n.d.). 2D convolution in hardware. <https://sistenix.com/sobel.html>
3. Lauri Vösandi, lauri.vosandi@gmail. com. (n.d.). Introduction. Lauri’s blog | Piping OV7670 video to VGA output on ZYBO. <https://lauri.xn--vsandi-pxa.com/hdl/zynq/zybo-ov7670-to-vga.html>

Materials from NUS CS2040C and the following papers have inspired and supported the use of Union-Find Disjoint Set algorithm for object detection in our use case:

1. Bochem, A., Kent, K. B., & Herpers, R. (2011). FPGA based real-time object detection approach with validation of precision and performance. *2011 22nd IEEE International Symposium on Rapid System Prototyping*, 9–15. <https://doi.org/10.1109/rsp.2011.5929969>
2. Olson, E. (2011). Apriltag: A robust and flexible visual fiducial system. *2011 IEEE International Conference on Robotics and Automation*, 3400–3407. <https://doi.org/10.1109/icra.2011.5979561>
3. Wang, J., & Olson, E. (2016). Apriltag 2: Efficient and robust fiducial detection. *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 4193–4198. <https://doi.org/10.1109/iros.2016.7759617>

Course Feedback:

Concept of clock dividers should be taught in conjunction with clock enable signals, especially for practical evaluation, having to instantiate a new clock and crossing signals between clock domains is likely the biggest cause of confusion for people.